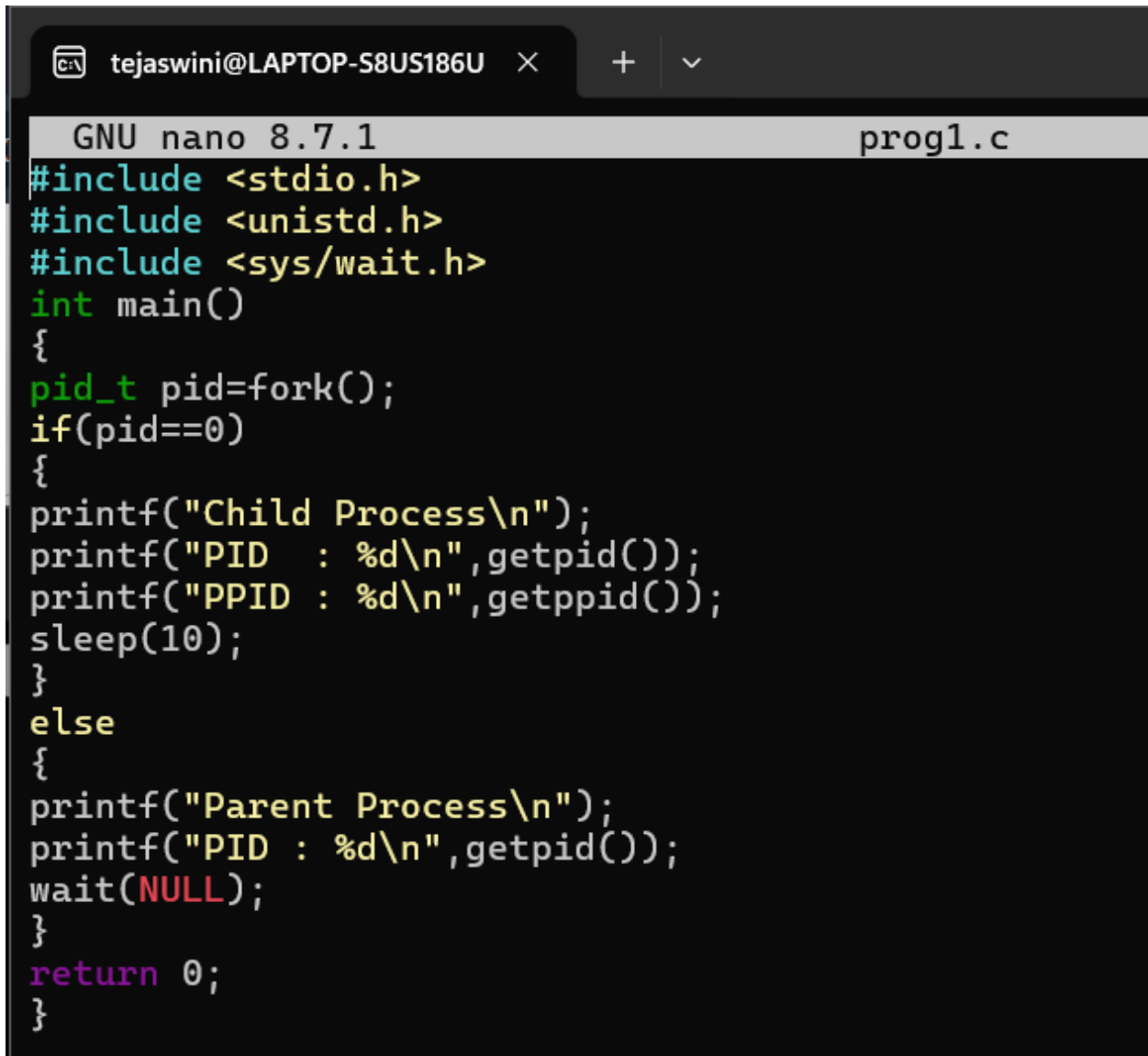


Name: P.Tejaswini
Rollno: 2520030011

Practical 3 & 4

Program-1:

A screenshot of a terminal window. The top bar shows the user 'tejaswini@LAPTOP-S8US186U' and the file 'prog1.c'. The terminal content shows a C program using 'fork()' to create a child process. The child process prints its PID and PPID and sleeps for 10 seconds. The parent process prints its PID and waits for the child to finish. The code is as follows:

```
GNU nano 8.7.1 prog1.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid=fork();
    if(pid==0)
    {
        printf("Child Process\n");
        printf("PID : %d\n",getpid());
        printf("PPID : %d\n",getppid());
        sleep(10);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n",getpid());
        wait(NULL);
    }
    return 0;
}
```

Output:

```
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ gcc prog1.c -o prog1
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ ./prog1
Parent Process
PID : 717
Child Process
PID : 718
PPID : 717
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ |
```

Program-2:

```
tejaswini@LAPTOP-S8US186U x + v
GNU nano 8.7.1 prog2.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int i;
    for(i=0;i<3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID=%d\n",i+1,getpid());
            return 0;
        }
    }
    for(i=0;i<3;i++)
    {
        wait(NULL);
    }
    printf("All children completed.\n");
    return 0;
}
```

Output:

```
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ nano prog2.c
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ gcc prog2.c -o prog2
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ ./prog2
Child 1 PID=784
Child 2 PID=785
Child 3 PID=786
All children completed.
tejaswini@LAPTOP-S8US186U:~/OS_Lab/prac_3_4$ |
```